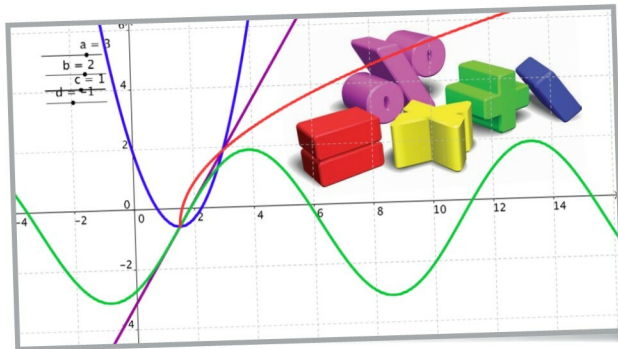


Try Out Polynomial Curve Fitting and Interpolation with Octave

This article goes a step further in our exploration of Octave, and describes how it can be used to do polynomial curve fitting and interpolation of data.



In various fields like physics, chemistry, statistics and economics, we very often come across the terms 'curve fitting' and 'interpolation'.

Given a set of data points from observation, we could determine the mathematical equation the data points satisfy. The curve fitting technique involves fitting the best possible curve through those points. Once the equation is determined, values at the points which have not been observed can be obtained by substituting in the equation. One may think of this as interpolation but this is not exactly so. Interpolation is restricted to finding unobserved values between two observed points only, using a pre-defined curve fit between those two points. With that, we may not be able to get values outside our range of observation. The main reason for using this is when we are not interested, or it is not meaningful or when we are unable to fit a known form of curve in the overall dataset but still want to get some approximations of values in between the observed data points.

Curve fitting

Let's assume that for any system, we have the following data points. For the inputs of 1, 3, 6, 10 and 20, we get the

outputs as 2.5, 7.5, 15.5, 24 and 45, respectively. Then, we would like to know how the input is related to the output. To get a feel of the relation, let's first plot these points on an x-y plane, say inputs as 'x' and outputs as 'y' (refer to Figure 1). The Octave code is as follows:

```
$ octave -qf
octave:1> x = [1 3 6 10 20];
octave:2> y = [2.5 7.5 15.5 24 45];
octave:3> plot(x, y, "r");
octave:4> xlabel("Inputs");
octave:5> ylabel("Outputs");
octave:6>
```

By observing the plots, the simplest relation that fits appears to be a straight line. So, let's try fitting a linear polynomial—a first order polynomial, i.e., a straight line. Here's the code to execute this:

```
octave:1> x = [1 3 6 10 20];
octave:2> y = [2.5 7.5 15.5 24 45];
octave:3> p = polyfit(x, y, 1)
```